

Ciencia de Datos

Aprendizaje no supervisado y métricas de desempeño

¿Qué es el aprendizaje no supervisado?

Un tipo de aprendizaje automático donde el modelo encuentra **patrones y estructuras** en los datos **sin etiquetas** previas.

- No existe una "respuesta correcta" conocida de antemano
- El algoritmo agrupa o reduce datos según su **similitud interna**
- Útil cuando etiquetar datos es costoso o imposible

Supervisado vs. no supervisado

Aprendizaje supervisado	Aprendizaje no supervisado
Datos etiquetados	Datos sin etiquetar
Predice una salida conocida	Descubre estructura oculta
Ej.: clasificación, regresión	Ej.: clustering, reducción de dimensionalidad
Se evalúa contra la verdad conocida	Se evalúa con métricas internas

K-means clustering

¿Cómo funciona K-means?

1. **Elegir K** — número de grupos (clusters) a formar
2. **Inicializar centroides** — puntos aleatorios como centros
3. **Asignar** — cada punto se une al centroide más cercano
4. **Actualizar** — se recalcula el centroide como el promedio del grupo
5. **Repetir** los pasos 3 y 4 hasta que los centroides ya no cambien

Objetivo: minimizar la distancia entre cada punto y su centroide asignado

K-means en Python

```
from sklearn.cluster import KMeans
import pandas as pd

# Datos de ejemplo: gasto anual y frecuencia de compra
X = df[['gasto_anual', 'frecuencia_compra']]

# Entrenar el modelo con 3 clusters
modelo = KMeans(n_clusters=3, random_state=42, n_init=10)
modelo.fit(X)

df['cluster'] = modelo.labels_
print(modelo.cluster_centers_)
```

 **Nota:** el parámetro `n_init` ejecuta el algoritmo varias veces con distintos centroides iniciales para evitar mínimos locales.

¿Cómo elegir el valor de K?

- **Método del codo (Elbow Method)** — grafica la inercia contra distintos valores de K y busca el "codo" de la curva
- **Coeficiente de silueta** — mide qué tan bien separado está cada punto de otros clusters
- No existe un K "correcto" universal: depende del contexto del problema

```
from sklearn.metrics import silhouette_score  
  
score = silhouette_score(X, modelo.labels_)  
print(f"Coeficiente de silueta: {score:.3f}")
```

Métricas de desempeño

¿Por qué medir el desempeño?

- Un modelo no sirve de nada si no sabemos **qué tan bien predice**
- Distintos problemas requieren **distintas métricas**
- Elegir mal la métrica puede llevar a conclusiones engañosas

Hoy veremos: precisión, recall, F1-score, MSE, ROC-AUC y entropía cruzada

Matriz de confusión

	Predicho: Positivo	Predicho: Negativo
Real: Positivo	Verdadero Positivo (VP)	Falso Negativo (FN)
Real: Negativo	Falso Positivo (FP)	Verdadero Negativo (VN)

Toda métrica de clasificación se construye a partir de esta tabla.

Precisión, Recall y F1-score

 **Precisión** = $VP / (VP + FP)$ — de lo que predije como positivo, ¿cuánto acerté?

 **Recall** = $VP / (VP + FN)$ — de todos los positivos reales, ¿cuántos detecté?

 **F1-score** = $2 \times (Precisión \times Recall) / (Precisión + Recall)$ — balance entre ambas

Métricas en Python

```
from sklearn.metrics import (  
    precision_score, recall_score, f1_score, mean_squared_error  
)  
  
y_real = [1, 0, 1, 1, 0, 1]  
y_pred = [1, 0, 0, 1, 0, 1]  
  
print("Precisión:", precision_score(y_real, y_pred))  
print("Recall:", recall_score(y_real, y_pred))  
print("F1-score:", f1_score(y_real, y_pred))
```

MSE — Error Cuadrático Medio

Usado en problemas de **regresión**, no de clasificación.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Penaliza más los errores grandes (por el cuadrado)
- Entre más cercano a 0, mejor el modelo

```
mse = mean_squared_error(y_real_continuo, y_pred_continuo)
```

Curva ROC y AUC

- La **curva ROC** grafica la tasa de verdaderos positivos contra la tasa de falsos positivos en distintos umbrales
- El **AUC** (área bajo la curva) resume el desempeño en un solo número

Valor de AUC	Interpretación
1.0	Clasificador perfecto
0.5	Equivalente a adivinar al azar
< 0.5	Peor que el azar

Entropía cruzada (Cross-Entropy)

Métrica usada para medir qué tan lejos está la **probabilidad predicha** de la clase real.

- Muy usada como función de pérdida en clasificación y redes neuronales
- Penaliza fuertemente las predicciones **seguras pero equivocadas**

```
from sklearn.metrics import log_loss  
perdida = log_loss(y_real, probabilidades_predichas)
```

Selección de métricas según el problema

¿Qué métrica usar?

Situación	Métrica recomendada
Clases desbalanceadas	Recall o F1-score
Falsos positivos son costosos	Precisión
Falsos negativos son costosos	Recall
Comparar clasificadores en general	ROC-AUC
Predicción de valores numéricos	MSE
Modelos probabilísticos / redes neuronales	Entropía cruzada

Casos reales

- **Bancos** priorizan **recall** al detectar fraude: prefieren revisar transacciones de más antes que dejar pasar un fraude
- **Correo spam** prioriza **precisión**: es peor marcar un correo importante como spam que dejar pasar algo de spam
- **Diagnóstico médico** combina **recall alto** con revisión humana posterior
- **Sistemas de recomendación** usan clustering (K-means) para segmentar usuarios con gustos similares

Para esta semana

- [] Explicar el funcionamiento del algoritmo K-means paso a paso
- [] Aplicar K-means en Python para segmentar un conjunto de datos
- [] Calcular e interpretar precisión, recall y F1-score
- [] Diferenciar cuándo usar MSE, ROC-AUC o entropía cruzada
- [] Reflexionar: **¿qué métrica elegirías para un modelo que detecta enfermedades?**

Próxima sesión

Formulación y ejecución · hipótesis nula · error tipo I y tipo II · validación de modelos

 **Evaluación Unidad III:** Portafolio — proyecto de ML con problemática real, métricas y prueba de hipótesis